KCPC Week 4 Papers
Foundations of Graph Theory, Traversals, and Structural Algorithms

King's Competitive Programming Club
Curated by Mehmet Kutay Bozkurt (@mkutay)

This annotated bibliography compiles the foundational research papers behind Workshop 4 (*Introduction to Graphs*). Each entry provides a complete citation, a persistent link or archive location, and the exact sections and page numbers relevant to the workshop slides and problem sheet. For textbook introductions and competitive programming tutorials, see *Week 4 Reading*.

I. Origins of Graph Theory and Degree Invariants

[1] Euler, L. (1736 / published 1741)

Solutio problematis ad geometriam situs pertinentis. Commentarii Academiae Scientiarum Imperialis Petropolitanae, 8, 128–140. https://scholarlycommons.pacific.edu/euler-works/53/

Where / why: Paragraphs 1–21 (pp. 128–140) formulate the Seven Bridges of Königsberg by stripping away metric geometry and retaining only discrete vertices (landmasses) and edges (bridges). Paragraphs 11–20 establish how vertex degrees govern walks and parity ($\sum_{v \in V} \deg(v) = 2|E|$). This paper is the historical origin of graph theory and underpins the degree-characterisation technique used in Slide Section 1 and Problem 2.2 (*Cyclic Components*).

II. Breadth-First Search and Shortest Unweighted Paths

[2] Moore, E. F. (1959)

The shortest path through a maze. In *Proceedings of an International Symposium on the Theory of Switching* (Harvard University, April 2–5, 1957), Part II, The Annals of the Computation Laboratory of Harvard University, Vol. 30, pp. 285–292. Harvard University Press.

Where / why: Pages 285–288 introduce Breadth-First Search (Algorithms A and B) as a wavefront expansion across unweighted graphs and prove that labelling vertices in non-decreasing distance layers guarantees a minimum-edge path, reconstructed by tracing predecessor steps backward from the destination. Directly underpins Slide Section 3, Problem 1.2 (*Message Route*), and Problem 3.1 (*Rudolf and Subway*).

[3] Lee, C. Y. (1961)

An algorithm for path connections and its applications. IRE Transactions on Electronic Computers, EC-10(3), 346–365. https://doi.org/10.1109/TEC.1961.5219222

Where / why: Sections I–II (pp. 346–351) independent of Moore formalise wave-propagation breadth-first search and parent back-tracing in discrete routing networks, proving optimality of the reconstructed path.

III. Depth-First Search, Linear-Time Representations, and Edge Classification

[4] Tarjan, R. E. (1972)

Depth-first search and linear graph algorithms. SIAM Journal on Computing, 1(2), 146–160. https://doi.org/10.1137/0201010

Where / why: Section 2 (pp. 146–150) formalises recursive Depth-First Search on adjacency lists, defines entry/exit vertex numbering, proves the $O(|V| + |E|)$ time bound, and establishes the structure of the DFS palm tree—showing that in an undirected graph every non-tree edge is a back-edge (frond) between an ancestor and a descendant, while directed graphs also admit cross and forward arcs. Directly underpins Slide Sections 2 and 4, Problem 1.1 (*Building Roads*), and Problem 1.3 (*Round Trip*).

[5] Hopcroft, J., & Tarjan, R. (1973)

Algorithm 447: Efficient algorithms for graph manipulation. Communications of the ACM, 16(6), 372–378. https://doi.org/10.1145/362248.362272

Where / why: Pages 372–375 detail concrete linear-space adjacency-list storage and $O(V+E)$ DFS routines for decomposing graphs into connected and biconnected components and extracting fundamental cycles.

IV. Bipartite Graphs and 2-Colouring

[6] Kőnig, D. (1916)

Über Graphen und ihre Anwendung auf Determinantentheorie und Mengenlehre. Mathematische Annalen, 77(4), 453–465. https://doi.org/10.1007/BF01456961

Where / why: Section 1, Theorem 2 (pp. 453–456) proves the foundational bipartite characterisation theorem: a graph is 2-colourable (bipartite) if and only if it contains no cycle of odd length. Directly underpins Slide Pattern 3, Problem 1.4 (*Building Teams*), and the auxiliary bipartite graph construction in Problem 3.1 (*Rudolf and Subway*).

V. Topological Sorting, DAGs, and Tree Diameter

[7] Kahn, A. B. (1962)

Topological sorting of large networks. Communications of the ACM, 5(11), 558–562. https://doi.org/10.1145/368996.369025

Where / why: Pages 558–560 present Kahn's in-degree algorithm: repeatedly extracting vertices of in-degree 0 and decrementing the in-degrees of their successors produces a topological ordering of any Directed Acyclic Graph (DAG) in linear time, and detects directed cycles when non-extracted vertices remain. Directly underpins Slide Pattern 5, Problem 2.3 (*DAG Number of Routes*), Problem 2.4 (*Longest Path*), and cycle/tree separation in functional graphs.

[8] Handler, G. Y. (1973)

Minimax location of a facility in an undirected tree graph. Transportation Science, 7(3), 287–293. https://doi.org/10.1287/trsc.7.3.287

Where / why: Section 1 (pp. 287–290) proves the two-sweep tree diameter theorem (originally observed for unweighted trees by Dijkstra around 1960): starting from an arbitrary vertex r in a tree, any vertex u at maximum distance from r is an endpoint of a longest path, so a second traversal from u finds the tree diameter in $O(V)$ time without dynamic programming. Directly underpins Slide Pattern 4 and Problem 2.5 (*Tree Diameter*).